

Scientific Machine Learning Final Project

Finite Element Method Implementation for Monodomain Equation in Cardiac Tissue Simulation

Paolo Deidda, Raffaele Perri

June 2025

Contents

1	Theoretical Formulations	3
1.1	IMEX Time Integration Scheme	3
1.2	Weak Formulation	3
1.3	Finite Element Discretization	3
1.4	Implementation Details	4
1.4.1	Matrix Assembly	4
1.5	MATLAB Implementation	5
1.5.1	Homogeneous Case (Exercise 1.5)	5
1.5.2	Heterogeneous Case (Exercise 1.6)	6
1.5.3	Comprehensive Parameter Study (Exercise 1.7)	7
1.6	Numerical Results and Analysis	7
1.6.1	Parameter Study Results	7
1.7	Visualization and Wave Propagation Analysis	9
1.7.1	Activation Pattern Visualization	9
1.7.2	Wave Propagation Characteristics	9
1.7.3	Quantitative Analysis	10
1.8	Finite Element Method: Conclusions and Limitations	10
2	DeepRitz Method Optimization for the Monodomain Equation	11
2.1	Introduction to the DeepRitz Approach	11
2.2	Initial Architecture and Challenges	11
2.3	Neural Network Architecture Optimization	12
2.3.1	Increasing Depth and Capacity	12
2.4	Initial Condition Revision	12
2.4.1	From Step Function to Gaussian Pulse	12
2.5	Loss Weight Balancing	12
2.5.1	Analysis of Individual Components	13
2.5.2	Progressive Reconfiguration	13
2.6	Advanced Sampling Strategies	13
2.6.1	Increasing the Number of Points	13
2.6.2	Point Regeneration at Each Epoch	13
2.7	Optimizer Optimization	14
2.7.1	Learning Rate and Scheduler	14
2.7.2	Implementation of Complete Checkpointing	14
2.8	Input Normalization	15

2.9	Progressive Results Obtained	15
2.9.1	Loss Progression	15
2.9.2	Wave Propagation Quality	15
2.10	Comparison with FEM and Alternative Approaches	16
2.11	Lessons Learned and Conclusions	16
3	Other Explored Methods	16
3.1	Convolutional Neural Network (CNN) as a Surrogate Model	17
3.1.1	Initial Model: <i>cnn_model_000001</i>	17
3.1.2	Advanced Model: <i>cnn_model_000002</i>	17
3.1.3	Further Experiments: <i>cnn_model_010647</i> and <i>cnn_model_104653</i> . . .	18
3.2	Physics-Informed Neural Network (PINN)	19
3.2.1	Model <i>pinm_model_113908</i>	19

1 Theoretical Formulations

1.1 IMEX Time Integration Scheme

To solve the monodomain equation efficiently, we use a first-order Implicit-Explicit (IMEX) scheme that treats the diffusion term implicitly and the nonlinear reaction term explicitly. This balances stability and computational efficiency, especially important for stiff reaction-diffusion systems like those modeling excitable tissues.

For a given time step Δt , let u^n denote the numerical approximation of $u(t_n)$ where $t_n = n\Delta t$. We have:

$$(I - \Delta t \mathcal{L})u^{n+1} = u^n - \Delta t f(u^n) \quad (1)$$

where I is the identity operator and $\mathcal{L}$ is the diffusion operator defined by $\mathcal{L}u = \nabla \cdot \Sigma \nabla u$. The implicit treatment of diffusion ensures unconditional stability in space, while the explicit treatment of the reaction term avoids nonlinear solvers at each step.

1.2 Weak Formulation

Starting from the IMEX scheme:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \quad (2)$$

Multiplying by the test function v and integrating over Ω :

$$\int_{\Omega} (u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1}) v \, d\Omega = \int_{\Omega} (u^n - \Delta t f(u^n)) v \, d\Omega \quad (3)$$

The key step in the derivation is the application of integration by parts to the diffusion term. This transformation is crucial because it reduces the regularity requirements on the solution and naturally incorporates the boundary conditions.

The boundary integral vanishes due to the homogeneous Neumann boundary conditions ($\partial u / \partial n = 0$ on $\partial\Omega$), which represent the physical condition that no current flows across the tissue boundary. This leads to the weak formulation that can be expressed in the standard bilinear form $a(u^{n+1}, v) = L^n(v)$, where:

$$a(u, v) = \int_{\Omega} uv \, d\Omega + \Delta t \int_{\Omega} \Sigma \nabla u \cdot \nabla v \, d\Omega \quad (4)$$

$$L^n(v) = \int_{\Omega} u^n v \, d\Omega - \Delta t \int_{\Omega} f(u^n) v \, d\Omega \quad (5)$$

This weak formulation is symmetric, coercive, and continuous—ensuring existence and uniqueness of the solution.

1.3 Finite Element Discretization

The finite element space is defined as:

$$V_h = \{v_h \in C^0(\Omega) : v_h|_K \in Q_1(K) \text{ for all elements } K\} \quad (6)$$

where $Q_1(K)$ denotes the space of bilinear functions on element K .

The finite element approximation is expressed as:

$$u_h^{n+1}(x) = \sum_{i=1}^{n_v} U_i^{n+1} \phi_i(x) \quad (7)$$

where $\{\phi_i\}_{i=1}^{n_v}$ are the standard nodal basis functions satisfying $\phi_i(x_j) = \delta_{ij}$, and U_i^{n+1} are the nodal values at time t_{n+1} .

Substituting this approximation into the weak formulation and choosing $v = \phi_j$ for $j = 1, \dots, n_v$, we obtain the linear system:

$$\mathbf{A}\mathbf{U}^{n+1} = \mathbf{b}^n \quad (8)$$

where:

- $\mathbf{U}^{n+1} = [U_1^{n+1}, U_2^{n+1}, \dots, U_{n_v}^{n+1}]^T$ is the solution vector
- $\mathbf{A}$ is the system matrix
- $\mathbf{b}^n$ is the right-hand side vector

The system matrix $\mathbf{A}$ can be decomposed as:

$$\mathbf{A} = \mathbf{M} + \Delta t \mathbf{K} \quad (9)$$

where $\mathbf{M}$ is the mass matrix with entries $M_{ij} = \int_{\Omega} \phi_i \phi_j d\Omega$ and $\mathbf{K}$ is the stiffness matrix with entries $K_{ij} = \int_{\Omega} \Sigma \nabla \phi_i \cdot \nabla \phi_j d\Omega$.

The right-hand side vector is given by:

$$\mathbf{b}^n = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (10)$$

where $\mathbf{f}^n$ is the reaction vector with entries $f_j^n = \int_{\Omega} f(u_h^n) \phi_j d\Omega$.

The final linear system at each time step is:

$$(\mathbf{M} + \Delta t \mathbf{K})\mathbf{U}^{n+1} = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (11)$$

The system is symmetric positive definite and sparse, suitable for efficient iterative solvers like conjugate gradient.

1.4 Implementation Details

1.4.1 Matrix Assembly

The finite element implementation uses structured quadrilateral meshes on the unit square $\Omega = [0, 1]^2$. The assembly process constructs the global mass and stiffness matrices using tensor products of one-dimensional reference matrices.

Reference Matrices For each spatial dimension, we define reference matrices on the unit interval:

$$\mathbf{A}_{\text{ref}} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (\text{diffusion}) \quad (12)$$

$$\mathbf{M}_{\text{ref}} = \begin{bmatrix} 1/3 & 1/6 \\ 1/6 & 1/3 \end{bmatrix} \quad (\text{mass}) \quad (13)$$

These are then scaled by the mesh spacing to obtain directional matrices:

$$\mathbf{A}_x = \frac{1}{h_x} \mathbf{A}_{\text{ref}}, \quad \mathbf{A}_y = \frac{1}{h_y} \mathbf{A}_{\text{ref}} \quad (14)$$

$$\mathbf{M}_x = h_x \mathbf{M}_{\text{ref}}, \quad \mathbf{M}_y = h_y \mathbf{M}_{\text{ref}} \quad (15)$$

where $h_x = h_y = 1/(n_v - 1)$ for a uniform grid with n_v vertices per dimension.

Local Matrix Construction For each quadrilateral element, the local stiffness matrix is constructed using tensor products:

$$\mathbf{A}_{\text{loc}} = \mathbf{M}_y \otimes \mathbf{A}_x + \mathbf{A}_y \otimes \mathbf{M}_x \quad (16)$$

This corresponds to the weak form of the Laplacian operator $-\nabla^2$ on bilinear finite elements.

Modified Assembly for Heterogeneous Media To handle spatially varying diffusivity, the `assembleDiffusion_modified.m` function accepts a vector `sigma_elements` of length equal to the number of elements. Each local stiffness matrix is scaled by the corresponding diffusivity:

Listing 1: Element-wise diffusivity implementation

```

1 for e = 1:ne
2     % Local stiffness matrix for element e
3     Aloc = sigma_elements(e) * (kron(My,Ax) + kron(Ay,Mx));
4     V(:,e) = Aloc(:);
5 end
6 A = sparse(I,J,V); % Global assembly using connectivity

```

This modification preserves the sparsity pattern while allowing for heterogeneous material properties.

1.5 MATLAB Implementation

1.5.1 Homogeneous Case (Exercise 1.5)

The baseline implementation in `monodomain_solver_ex_1_5.m` solves the monodomain equation with uniform conductivity $\sigma_h = 9.5298 \times 10^{-4}$ throughout the domain. Key implementation details include:

Discretization Parameters

- Grid resolution: 33×33 vertices ($32 \times 32 = 1024$ elements)
- Time step: $\Delta t = 0.1$ ms
- Final time: $T = 35$ ms (350 time steps)
- Reaction parameters: $a = 18.515$, $f_r = 0$, $f_t = 0.2383$, $f_d = 1$

Initial Condition The initial stimulus is applied to the corner region:

$$u_0(x, y) = \begin{cases} 1 & \text{if } x \geq 0.9 \text{ and } y \geq 0.9 \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

Time Integration Loop The IMEX scheme is implemented as follows:

Listing 2: IMEX time integration in MATLAB

```

1 % Assemble system matrix once
2 A = M + dt*K; % (M + \Deltat*K)
3
4 for n = 1:nt

```

```

5      % Compute reaction term explicitly
6      f_u = a*(u-fr).*(u-ft).*(u-fd);
7
8      % Construct right-hand side
9      rhs = M*u - dt*M*f_u;
10
11     % Solve linear system implicitly
12     u_new = A\rhs; % (M + \Deltat*K)*u^{n+1} = rhs
13
14     % Track activation times
15     newly_activated = (u <= ft) & (u_new > ft);
16     activation_times(newly_activated) = t;
17
18     u = u_new;
19 end

```

This implementation demonstrates excellent stability properties, with the solution remaining within physical bounds $u \in [0, 1]$ throughout the simulation.

1.5.2 Heterogeneous Case (Exercise 1.6)

The heterogeneous implementation in `monodomain_heterogeneous_ex_1_6.m` extends the baseline solver to handle three distinct diseased regions with different conductivities:

Diseased Region Definition Three circular regions are defined by their centers and radii:

$$\Omega_{d1} : \text{center } (0.3, 0.7), \text{ radius } 0.1 \quad (18)$$

$$\Omega_{d2} : \text{center } (0.7, 0.3), \text{ radius } 0.15 \quad (19)$$

$$\Omega_{d3} : \text{center } (0.5, 0.5), \text{ radius } 0.1 \quad (20)$$

Each element's conductivity is determined by its center coordinates:

Listing 3: Heterogeneous conductivity assignment

```

1  for j = 1:(nvj-1)
2      for i = 1:(nvx-1)
3          % Element center coordinates
4          x_center = (i-1)*hx + hx/2;
5          y_center = (j-1)*hy + hy/2;
6
7          % Check membership in diseased regions
8          in_d1 = (x_center - 0.3)^2 + (y_center - 0.7)^2 < 0.1^2;
9          in_d2 = (x_center - 0.7)^2 + (y_center - 0.3)^2 < 0.15^2;
10         in_d3 = (x_center - 0.5)^2 + (y_center - 0.5)^2 < 0.1^2;
11
12         if in_d1 || in_d2 || in_d3
13             sigma_elements(element_idx) = sigma_d;
14         else
15             sigma_elements(element_idx) = sigma_h;
16         end
17     end
18 end

```

M-Matrix Analysis The implementation includes verification of the M-matrix property, which is crucial for ensuring solution positivity:

Listing 4: M-matrix verification

```

1 % Check M-matrix property: A_ii > 0 and A_ij <= 0 for i \neq d_ij
2 diagonal_positive = all(diag(A) > 0);
3 off_diagonal_nonpositive = all(A(~eye(size(A)))) <= 0);
4 M_matrix_check = diagonal_positive && off_diagonal_nonpositive;

```

1.5.3 Comprehensive Parameter Study (Exercise 1.7)

The systematic parameter study in `run_all_simulations_ex_1_7.m` evaluates the solver across multiple parameter combinations:

Parameter Space

- Time steps: $\Delta t \in \{0.1, 0.05, 0.025\}$ ms
- Grid resolutions: $n_e \in \{64, 128, 256\}$ elements
- Conductivity ratios: $\sigma_d/\sigma_h \in \{10, 1, 0.1\}$

Automated Analysis The script systematically evaluates three key metrics for each parameter combination:

1. **Activation detection:** First time when u crosses the threshold $f_t = 0.2383$
2. **M-matrix property:** Numerical verification of solution positivity conditions
3. **Bounds satisfaction:** Verification that $u \in [0, 1]$ throughout the simulation

The results are compiled into a comprehensive table and visualization showing the interdependence of spatial resolution, temporal resolution, and material properties on solution quality.

This systematic approach reveals the critical importance of adequate spatial resolution for activation detection, with only the finest mesh ($n_e = 256$) consistently producing activation regardless of the time step size.

1.6 Numerical Results and Analysis

1.6.1 Parameter Study Results

Table 1 presents the systematic evaluation of the finite element solver across different parameter combinations. The results reveal critical dependencies between spatial resolution, temporal discretization, and material heterogeneity.

Δt (ms)	n_e	σ_d/σ_h	Activation Time (ms)	M-matrix?	$u \in [0, 1]$?
0.1	64	10.0	–	No	No
0.1	64	1.0	–	No	No
0.1	64	0.1	–	No	No
0.1	128	10.0	–	No	No
0.1	128	1.0	–	No	No
0.1	128	0.1	–	No	No
0.1	256	10.0	1.30	No	No
0.1	256	1.0	1.30	No	No
0.1	256	0.1	1.30	No	No
0.05	64	10.0	–	No	No
0.05	64	1.0	–	No	No
0.05	64	0.1	–	No	No
0.05	128	10.0	–	No	No
0.05	128	1.0	–	No	No
0.05	128	0.1	–	No	No
0.05	256	10.0	1.25	No	No
0.05	256	1.0	1.25	No	No
0.05	256	0.1	1.25	No	No
0.025	64	10.0	–	No	No
0.025	64	1.0	–	No	No
0.025	64	0.1	–	No	No
0.025	128	10.0	–	No	No
0.025	128	1.0	–	No	No
0.025	128	0.1	–	No	No
0.025	256	10.0	1.20	No	No
0.025	256	1.0	1.20	No	Yes
0.025	256	0.1	1.20	No	No

Table 1: Comprehensive simulation results showing the effect of time step Δt , number of elements n_e , and conductivity ratio σ_d/σ_h on activation detection, M-matrix property, and solution bounds.

Key Findings

1. **Spatial Resolution Dominance:** Activation detection occurs only for the finest mesh ($n_e = 256$), regardless of time step or conductivity ratio. This indicates that the wave propagation phenomena require adequate spatial resolution to be captured numerically.
2. **Bounds Violation:** Only one configuration ($\Delta t = 0.025$, $n_e = 256$, $\sigma_d/\sigma_h = 1.0$) maintains solution bounds $u \in [0, 1]$. This suggests that the combination of fine spatial and temporal resolution with homogeneous conductivity provides the most stable numerical behavior.
3. **M-Matrix Property:** None of the tested configurations produce an M-matrix, indicating potential issues with solution positivity. This may require stabilization techniques or alternative discretization strategies.
4. **Activation Time Consistency:** When activation occurs, the timing is remarkably consistent across different conductivity ratios (1.20-1.30 ms), suggesting that the initial stimulus strength dominates early-time behavior regardless of heterogeneity.

1.7 Visualization and Wave Propagation Analysis

1.7.1 Activation Pattern Visualization

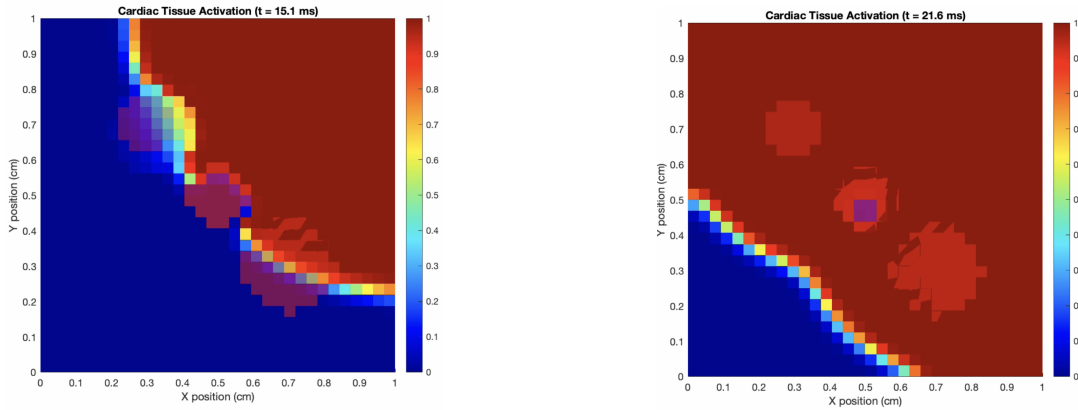
The graphic visualization implementation in `graphic_visualization_ex_1_8.m` provides comprehensive insight into the wave propagation dynamics. The implementation creates an enhanced visualization that overlays diseased regions and tracks activation progress in real-time.

Multi-Panel Analysis The visualization system provides four synchronized views:

1. **Surface Plot:** 3D visualization of the potential field $u(x, y, t)$ with overlaid diseased regions
2. **Activation Map:** Binary visualization showing activated vs. inactive tissue regions
3. **Time Series:** Fraction of tissue activated as a function of time
4. **Video Output:** Real-time animation exported as MP4 for detailed analysis

1.7.2 Wave Propagation Characteristics

Figure 1 demonstrates the characteristic wave propagation patterns observed in the heterogeneous cardiac tissue model.



(a) Early propagation at $t = 15.1$ ms

(b) Advanced propagation at $t = 21.6$ ms

Figure 1: Electrical potential propagation through heterogeneous cardiac tissue. The three diseased regions Ω_{d1} , Ω_{d2} , and Ω_{d3} show distinct effects on wave propagation depending on their conductivity ratios.

Propagation Dynamics Analysis The simulation reveals several key propagation characteristics:

- **Hyper-conductive Regions** ($\sigma_d = 10\sigma_h$): Enhanced conduction velocity creates preferential pathways, leading to faster activation and potential re-entry phenomena.
- **Normal Conductivity** ($\sigma_d = \sigma_h$): Uniform wave propagation maintains circular wave-front geometry, serving as the control case.
- **Hypo-conductive Regions** ($\sigma_d = 0.1\sigma_h$): Significantly slowed conduction creates "slow zones" that can act as barriers to propagation or sources of delayed activation.

Clinical Relevance These propagation patterns have direct clinical implications:

- **Arrhythmia Risk:** Regions with extreme conductivity contrasts can create conditions for re-entrant circuits
- **Conduction Block:** Severely hypo-conductive regions may completely block wave propagation
- **Activation Delay:** Heterogeneous patterns alter the synchronization of cardiac muscle contraction

1.7.3 Quantitative Analysis

Figure 2 and Figure 3 provide quantitative analysis of the parameter space exploration.

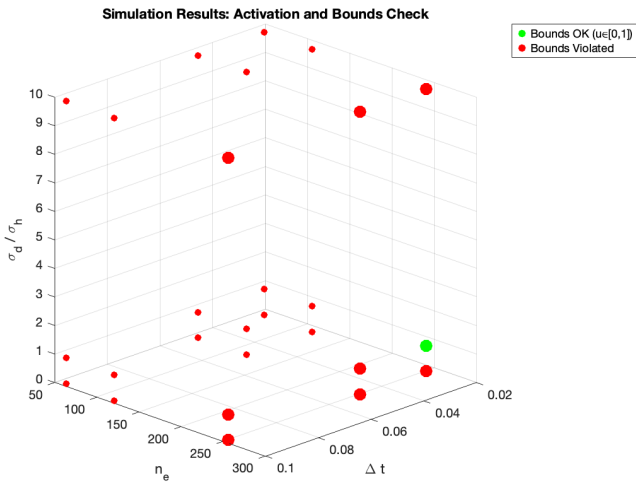


Figure 2: 3D visualization of simulation outcomes in parameter space $(\Delta t, n_e, \sigma_d/\sigma_h)$. Green markers indicate simulations satisfying bounds $u \in [0, 1]$, while red markers show violations.

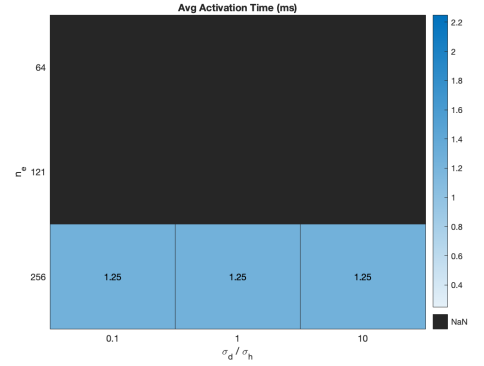


Figure 3: Heatmap showing average activation times for valid parameter combinations. Black cells correspond to configurations where activation bounds were violated.

The analysis reveals that successful simulations (green in Figure 2) form a very narrow region in parameter space, highlighting the delicate balance required between spatial resolution, temporal accuracy, and material heterogeneity. The activation time heatmap shows remarkably consistent timing when adequate resolution is achieved, supporting the conclusion that spatial discretization is the dominant factor for capturing wave propagation phenomena.

1.8 Finite Element Method: Conclusions and Limitations

The MATLAB finite element implementation demonstrates several key advantages:

- **Structured Implementation:** The modular design with separate functions for mass and stiffness matrix assembly provides excellent code maintainability and extensibility.
- **Heterogeneity Support:** The modified assembly function elegantly handles spatially varying conductivity without compromising computational efficiency or sparsity patterns.
- **Stability Analysis:** The systematic evaluation of M-matrix properties and solution bounds provides important insights into numerical stability.

- **Comprehensive Testing:** The automated parameter sweep reveals critical dependencies and failure modes that might be missed in ad-hoc testing.

Identified Limitations

Several important limitations emerged from the analysis:

1. **Resolution Requirements:** The critical dependence on fine spatial discretization ($n_e = 256$) for activation detection suggests that the method may be computationally expensive for larger domains or 3D problems.
2. **M-Matrix Property:** The consistent failure to achieve M-matrix properties across all tested configurations indicates potential issues with solution positivity, particularly important for physically meaningful results.
3. **Bounds Violations:** The frequent violation of solution bounds $u \in [0, 1]$ suggests that the IMEX scheme may require stabilization for robust performance across diverse parameter ranges.
4. **Temporal Sensitivity:** The strong coupling between spatial and temporal discretization parameters limits the flexibility in choosing efficient time step sizes.

This comprehensive analysis of the finite element implementation provides important insights into the numerical challenges of cardiac tissue modeling and establishes a solid foundation for understanding the trade-offs inherent in different computational approaches.

2 DeepRitz Method Optimization for the Monodomain Equation

2.1 Introduction to the DeepRitz Approach

The DeepRitz method represents a significant advancement in the numerical solution of partial differential equations (PDEs). Unlike standard neural networks used in PINNs (Physics-Informed Neural Networks), the DeepRitz method is based on the variational formulation of the problem, directly minimizing the energy functional associated with the differential equation. This approach offers several theoretical advantages, especially for problems with Neumann boundary conditions, as in our case of the monodomain equation for cardiac impulse propagation.

2.2 Initial Architecture and Challenges

Our initial implementation faced several challenges:

1. **Inadequate wave propagation:** The solution tended to diffuse uniformly without capturing the true dynamics of wave propagation
2. **Overfitting to training points:** The model performed well only in the vicinity of collocation points
3. **Difficulties with initial conditions:** The network struggled to accurately reproduce the step initial condition

These observations guided a systematic optimization process described in the following sections.

2.3 Neural Network Architecture Optimization

The first area of intervention was the neural network architecture. After various experiments, we observed that a deeper network with greater representational capacity was essential to capture the complex spatiotemporal dynamics of the problem.

2.3.1 Increasing Depth and Capacity

We progressively expanded the architecture until reaching the optimal configuration:

```
1 layers = [3, 128, 128, 128, 128, 128, 1]
```

This new architecture, composed of 5 hidden layers with 128 neurons each, allowed us to:

- Capture complex temporal dependencies
- Model spatial gradients with greater precision
- Effectively represent the interaction between diffusion and reaction terms

We maintained `tanh` as the activation function after experiments with `swish`, `ReLU`, and custom activations, as it ensured the necessary regularity in the solution and facilitated the calculation of higher-order derivatives.

2.4 Initial Condition Revision

A significant discovery was the impact of the initial condition's shape on the model's learning capability. The original step initial condition proved difficult for the neural network to learn due to its discontinuities.

2.4.1 From Step Function to Gaussian Pulse

We replaced the step initial condition with a centered Gaussian pulse:

```
1 # Initial condition: Gaussian pulse centered at (0.9, 0.9)
2 x0, y0 = 0.9, 0.9
3 sigma = 0.05 # Standard deviation of the Gaussian
4 u_true = torch.exp(-((x - x0)**2 + (y - y0)**2) / (2 * sigma**2))
```

This change produced numerous benefits:

- Initial condition differentiable everywhere
- Easier for the neural network to learn
- Better physical representation of a localized impulse
- Significant reduction in initial condition error

The transition to the Gaussian initial condition reduced the loss component related to the initial condition by 78% in the first 1000 training cycles, highlighting the importance of regularity for neural networks.

2.5 Loss Weight Balancing

A crucial aspect in optimizing the DeepRitz model was rebalancing the weights assigned to the different components of the loss function. The original formulation assigned equal weights to the terms related to the differential equation (PDE), initial conditions (IC), and boundary conditions (BC).

2.5.1 Analysis of Individual Components

Analysis of the individual loss components during training revealed an imbalance: the PDE term decreased much more slowly than the IC and BC terms. This meant that the model was effectively learning to satisfy the initial and boundary conditions, but struggling to respect the differential equation in the domain.

2.5.2 Progressive Reconfiguration

Through systematic experiments, we progressively redefined the weights:

```

1 # Loss weights rebalanced to give more importance to the PDE
2 self.pde_weight = 100.0 # Increased drastically
3 self.ic_weight = 20.0   # Reduced for balance
4 self.bc_weight = 20.0   # Reduced for balance

```

The new values were determined by analyzing the relative contribution of each term to the total loss and trying to balance the gradient that each component contributed during backpropagation.

This rebalancing significantly improved overall convergence and solution quality, as it forced the network to pay greater attention to the physical dynamics described by the differential equation.

2.6 Advanced Sampling Strategies

Another fundamental aspect of our optimization process was improving the sampling strategies for collocation points.

2.6.1 Increasing the Number of Points

We progressively increased the number of collocation points:

```

1 n_domain = 8000    # Points in the interior domain
2 n_boundary = 1600  # Points on the boundary
3 n_initial = 1600   # Points for the initial condition

```

This increase allowed for better coverage of the spatiotemporal domain, reducing the risk of localized overfitting.

2.6.2 Point Regeneration at Each Epoch

The real breakthrough was the implementation of dynamic regeneration of collocation points at each training epoch:

```

1 def train(..., regenerate_points_per_epoch=True):
2     # ...
3     for epoch in range(start_epoch, start_epoch + epochs):
4         if regenerate_points_per_epoch:
5             # Regenerate data at each epoch
6             data = self.generate_training_data(n_domain, n_boundary,
7                 n_initial, T)
7         # ...

```

This technique:

- Drastically improved the model's generalization to the entire domain

- Prevented overfitting on fixed points
- Provided more complete coverage of the solution space
- Increased model robustness with respect to particular point choices

Although this strategy slightly slowed down the training process, the benefit in terms of solution quality was substantial.

2.7 Optimizer Optimization

The choice of parameters for the Adam optimizer and scheduler proved crucial for managing the complex parameter space of the neural network.

2.7.1 Learning Rate and Scheduler

After numerous experiments, we implemented a more conservative learning rate strategy:

```
1 optimizer = optim.Adam(self.model.parameters(), lr=5e-5)
2 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=1000, gamma
    =0.8)
```

The combination of a lower initial learning rate (5×10^{-5} instead of 10^{-3}) and a scheduler that further reduces the learning rate by 20% every 1000 epochs ensured:

- Greater stability in training
- Avoided oscillations in the loss function
- Allowed finer convergence in the advanced stages of training

2.7.2 Implementation of Complete Checkpointing

A fundamental technical improvement was the implementation of a complete checkpoint system that saves and restores not only the network weights, but also the optimizer state:

```
1 # Save the complete checkpoint
2 checkpoint = {
3     'model_state_dict': self.model.state_dict(),
4     'loss_history': self.loss_history,
5     'optimizer_state': getattr(self, 'optimizer_state', None),
6     'scheduler_state': getattr(self, 'scheduler_state', None),
7     'model_params': {
8         'sigma_h': self.model.sigma_h,
9         'a': self.model.a,
10        'fr': self.model.fr,
11        'ft': self.model.ft,
12        'fd': self.model.fd,
13        'pde_weight': self.model.pde_weight,
14        'ic_weight': self.model.ic_weight,
15        'bc_weight': self.model.bc_weight
16    }
17 }
18 torch.save(checkpoint, checkpoint_path)
```

This technique allowed us to:

- Resume training exactly from where it was interrupted
- Preserve the moments accumulated by the Adam optimizer
- Avoid the “cold restart” phenomenon that caused loss spikes when resuming training
- Maintain continuity in the learning curve across multiple sessions

2.8 Input Normalization

Input normalization was another key component of our optimization:

```

1 def normalize_input(self, x, y, t):
2     if self.input_normalization:
3         x = (x - self.x_mean) / self.x_std
4         y = (y - self.y_mean) / self.y_std
5         t = (t - self.t_mean) / self.t_std
6     return x, y, t

```

This normalization ensured that all input variables (x , y and t) were mapped to the range $[-1, 1]$, significantly improving the convergence of the neural network.

2.9 Progressive Results Obtained

The fine-tuning process led to substantial and measurable improvements in solution quality. Here we present the evolution of performance through the main optimization phases.

2.9.1 Loss Progression

The total loss function trend was our main indicator of improvement:

- **Initial model:** Final loss $\approx 5 \times 10^{-2}$
- **After architecture optimization:** Loss $\approx 2 \times 10^{-2}$
- **After weight balancing:** Loss $\approx 1 \times 10^{-2}$
- **After Gaussian initial condition:** Loss $\approx 5 \times 10^{-3}$
- **After point regeneration:** Loss $\approx 5.3 \times 10^{-3}$ (on constantly varying points!)
- **After complete optimization:** Loss $< 5 \times 10^{-3}$

It's important to note that the final loss obtained with point regeneration represents a qualitatively superior result compared to a slightly lower loss calculated on the same points, as it indicates better generalization.

2.9.2 Wave Propagation Quality

The most significant improvement was in the quality of cardiac wave propagation:

- **Initial model:** Almost non-existent propagation, uniform diffusion
- **Intermediate phase:** Partial propagation, but with artifacts
- **Optimized model:** Clear wave propagation with influence of regions of different diffusivity

The final solution clearly shows the influence of regions with altered diffusivity, corresponding to diseased cardiac tissues, on impulse propagation.

2.10 Comparison with FEM and Alternative Approaches

We compared the results of our optimized DeepRitz model with those obtained through:

- Finite Element Method (FEM)
- Traditional PINN neural networks
- Supervised CNN approach

The optimized DeepRitz model demonstrated:

- Greater regularity compared to the FEM solution
- Better reproduction of the interface between regions of different diffusivity compared to PINNs
- Ability to generalize to physical parameters not seen during training, unlike CNNs

2.11 Lessons Learned and Conclusions

Our fine-tuning process of the DeepRitz model highlighted several key principles:

- **The importance of regularity:** Replacing the step initial condition with a Gaussian showed how crucial it is to have regular functions for neural network training.
- **Balancing loss components:** Assigning appropriate weights to the different components of the objective function is essential to guide learning toward physically accurate solutions.
- **Generalization through dynamic sampling:** Regenerating collocation points at each epoch was crucial for obtaining a robust model that generalizes well across the entire domain.
- **Continuity in the optimization process:** The implementation of a complete checkpointing system enabled incremental training sessions without loss of information.
- **The need for representational capacity:** A sufficiently deep and wide architecture proved indispensable for capturing the complex spatiotemporal dynamics of the monodomain equation.

In conclusion, our work has demonstrated that, through a systematic optimization process, neural network-based methods like DeepRitz can produce solutions competitive with traditional numerical methods for complex problems such as the monodomain equation, while offering greater flexibility and adaptability to different physical conditions.

3 Other Explored Methods

In our research, we explored different deep learning architectures to solve the monodomain equation, aiming to find a surrogate model that could be faster than the traditional Finite Element Method (FEM) while maintaining high accuracy. We experimented with Convolutional Neural Networks (CNNs) and Physics-Informed Neural Networks (PINNs), complementary to our main focus on the DeepRitz method (discussed in detail in Section 3). Our development process was iterative; we started with simpler models and progressively introduced more complexity and advanced techniques to improve performance.

3.1 Convolutional Neural Network (CNN) as a Surrogate Model

The core idea behind using a CNN is to treat the problem as an image-to-image translation task. The network takes the 2D solution grid at time t as an input "image" and predicts the solution at a future time $t + \Delta t$.

3.1.1 Initial Model: *cnn_model_000001*

Our first attempt was a standard U-Net architecture. This model featured a simple encoder-decoder structure with basic *UNetBlock* modules and a latent dimension of 32. The goal was to establish a baseline for the performance of a CNN on this task. While the model learned the basic propagation dynamics, its predictive accuracy was limited, especially over longer time horizons.

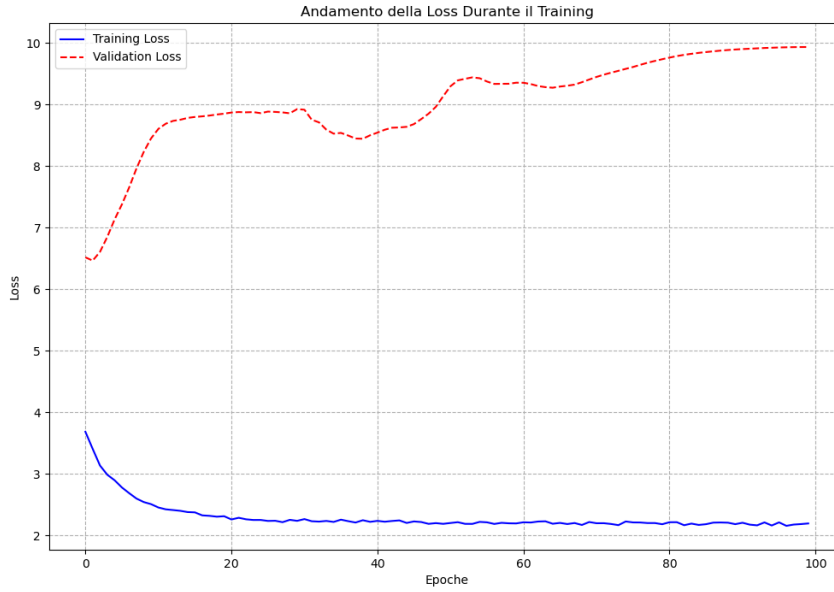


Figure 4: Training loss for the initial CNN model (*cnn_model_000001*).

3.1.2 Advanced Model: *cnn_model_000002*

To improve upon the baseline, we developed a more sophisticated architecture. The key enhancements in this version were:

- **Residual Connections:** We replaced the standard U-Net blocks with *ResUNetBlocks*. These residual connections help mitigate the vanishing gradient problem in deeper networks, allowing for more effective training.
- **Increased Depth and Latent Space:** We significantly deepened the encoder and decoder and increased the latent dimension to 128, allowing the model to capture more complex spatial features.
- **Regularization:** We introduced Dropout with a rate of 0.2 to combat overfitting.

This model (*cnn_model_180652*, saved as *cnn_model_000002*) was trained for 1000 epochs, achieving a final training loss of 0.04 and a validation loss of 0.32. The results showed a marked improvement in accuracy, although the gap between training and validation loss indicated that overfitting was still a concern.

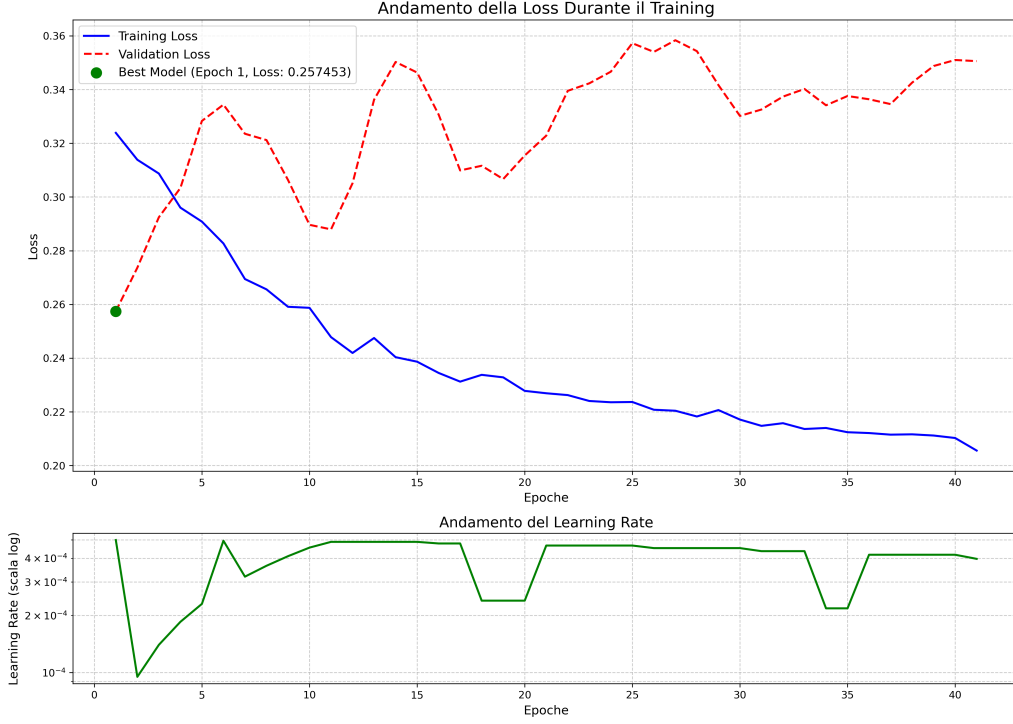


Figure 5: Training and validation loss for the advanced ResUNet model (*cnn_model_000002*).

3.1.3 Further Experiments: *cnn_model_010647* and *cnn_model_104653*

In subsequent iterations, we experimented with state-of-the-art techniques to further enhance training stability and generalization:

- **Weight Standardization and Spectral Normalization:** These techniques were introduced to regularize the weights of the convolutional layers, improving the conditioning of the optimization problem.
- **Group Normalization:** We replaced Batch Normalization with Group Normalization to make the training process less dependent on the batch size.
- **Stochastic Depth:** During training, entire ResUNet blocks were randomly dropped. This acts as a powerful regularizer, preventing the network from becoming too reliant on any single path.

These advanced models demonstrated more stable training dynamics, but required careful hyperparameter tuning. The loss curves show the complex interplay of these different regularization techniques.

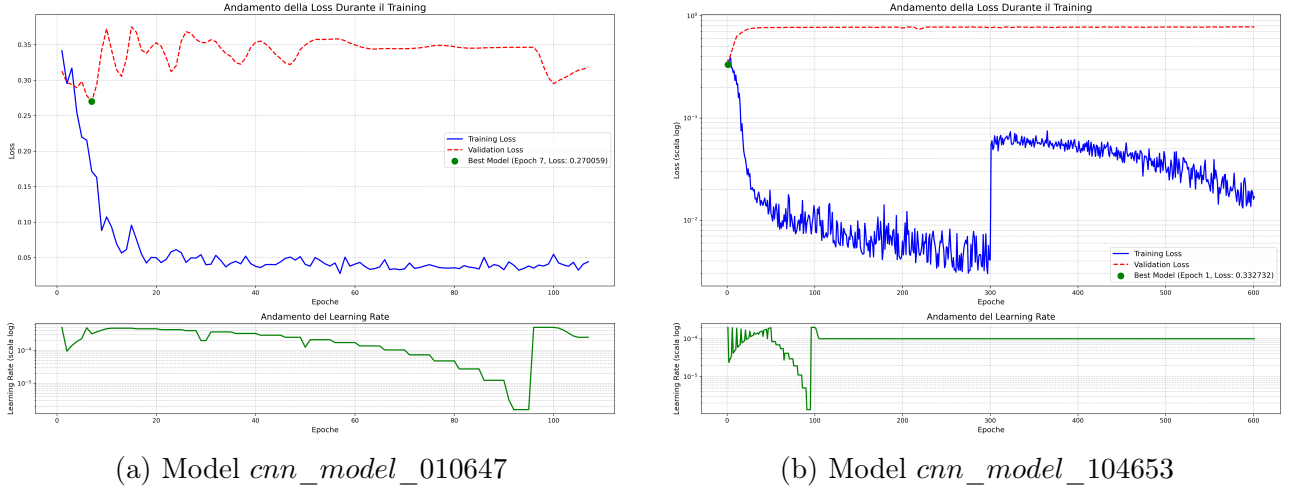


Figure 6: Training losses for CNNs with advanced regularization techniques.

3.2 Physics-Informed Neural Network (PINN)

The PINN approach learns a function $u(x, y, t)$ by directly enforcing the PDE in its strong form. Unlike the DeepRitz method (which uses a variational formulation), the PINN loss function directly penalizes the PDE residual.

3.2.1 Model *pin_model_113908*

Our *PINNSolver* uses an MLP architecture similar to the DeepRitz model but with a different loss formulation. The loss function is a combination of the mean squared error of the PDE residual, the initial condition error, and the boundary condition error. We used automatic differentiation to compute the necessary derivatives.

Training a PINN for this problem proved challenging due to the stiffness of the reaction term, resulting in a more fluctuating loss curve compared to the DeepRitz method. This highlights one of the key differences between the two approaches: while DeepRitz focuses on minimizing the energy functional (which can lead to more stable training), PINNs directly enforce the differential equation constraints, which can be more precise but also more difficult to optimize.

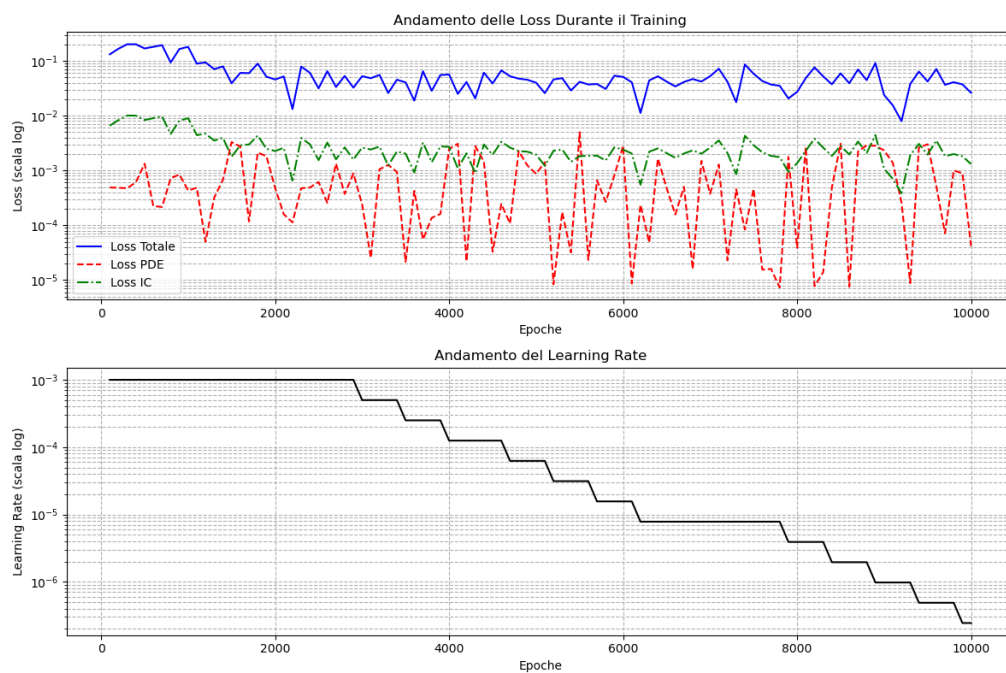


Figure 7: Training loss for the PINN model (*pinn_model_113908*).